

李明

(+86) 18515287309 | hynever@qq.com | Hangzhou, China

求职意向: Python 后端开发工程师 / 全栈工程师 (中级)

个人简介

5 年 Python 后端开发经验, 主导多个高并发、高可用微服务系统从 0 到 1 落地及迭代升级, 覆盖电商、SaaS、IoT 数据平台等领域。擅长**复杂业务建模、系统性能优化与 DevOps 实践**, 具备扎实的工程化能力与技术领导力。追求代码质量与可维护性, 倡导“可观测、可回滚、可扩展”的服务设计原则。

核心技术能力

类别	技术栈与能力深度
语言与框架	Python 3.8+ (精通)、Django (DRF 深度定制)、Flask/FastAPI (生产级)、SQLAlchemy Core/ORM (复杂查询优化)
系统设计	微服务架构 (领域拆分/服务治理)、高并发设计 (缓存/异步/限流)、CQRS 模式、事件驱动架构 (Kafka)
数据库	PostgreSQL (分区表/JSONB/全文检索)、MySQL (主从/读写分离)、Redis (Cluster/Stream/Lua 脚本)、MongoDB (文档建模)
中间件与云原生	RabbitMQ / Kafka / Celery (分布式任务)、Docker + Kubernetes (Helm 部署)、Prometheus + Grafana (指标监控)、ELK (日志分析)
工程实践	Git Flow / Code Review / CI-CD (GitLab CI + Argo CD)、TDD/BDD (pytest + behave)、Sentry/APM (Datadog)
协作与软技能	技术方案评审、跨团队协作 (产品/前端/测试)、技术文档撰写、新人带教

重点项目经历

🚀 企业级 SaaS 营销自动化平台（后端负责人） | 2023.04 – 至今

- ◆ 客户：某跨境电商服务商（年 GMV > ¥10 亿） / 团队：8 人（3 后端+2 前端+2 测试+1PM）
- ◆ 目标：重构老旧单体系统，支持千万级用户触达、毫秒级规则引擎、多租户隔离

🔧 架构与关键技术实现：

- 微服务拆分与治理

- 主导架构演进：单体（Django Monolith） → 分层微服务（用户中心/消息引擎/行为分析/租户管理）
- 服务通信：gRPC（核心高频调用） + REST（外部集成） + Kafka（事件解耦，日均消息 500 万+）
- 服务发现与熔断：Consul + Resilience4j（Python via Pyresilience），SLA 提升至 99.95%

- 高性能规则引擎（核心模块）

- 业务需求：支持“用户行为 + 标签 + 时间窗口”组合触发营销动作（如：加购 3 次未下单 → 发券）
- 自研 DSL + AST 解析器（lark-parser），规则配置化（JSON Schema + 前端可视化编排）
- 执行优化：
 - 热点规则预编译为 Python 字节码（compile + exec）
 - 状态缓存：Redis + Bloom Filter 降低 DB 查询（误判率 < 0.1%）
 - 并行计算：Celery + 分片（sharding_key=user_id%64），吞吐达

12K TPS

- 多租户与数据隔离

- 混合隔离策略：
 - Schema 隔离（核心客户）

- Row-level 隔离 (tenant_id 字段 + Row Security Policies in PostgreSQL)
- 缓存隔离: Redis Key 前缀 tenant:{id}:
- 租户级配额控制: Redis Token Bucket 实现 API 限流 (动态调整)
- 可观测性与运维
 - 全链路追踪: OpenTelemetry + Jaeger (Trace ID 透传)
 - 关键指标监控: 自定义 Prometheus Exporter (规则命中率、延迟分布、队列积压)
 - 自动化发布: GitLab CI → Helm → Argo CD (蓝绿部署, 回滚 < 30s)

✅ 业务价值:

- 支撑客户日均 800 万+ 触达任务, 峰值延迟 < 200ms (旧系统 > 2s)
- 运维成本下降 40% (自动化监控告警覆盖 100% 核心链路)
- 方案获公司年度“技术创新奖”

📦 IoT 设备数据中台 (核心开发者 → 技术 Owner) | 2021.08 – 2023.03

◆ 客户: 工业传感器厂商 / 数据规模: 日均 2 亿+ 设备消息, 峰值 5K msg/s

🔧 关键挑战与解决方案:

- 高吞吐数据接入层
 - 协议适配: MQTT (EMQX 集群) + HTTP Webhook + 自研二进制协议 (struct 解包)
 - 流量削峰: Kafka 分区扩容 (64 partitions) + 消费者组动态扩缩容 (K8s HPA)
 - 数据校验: Schema Registry (Avro) + 失败消息死信队列 (DLQ) 重试机制
- 实时计算与存储

- 实时看板: Flink PyFlink (Python UDF) 计算设备在线率/异常告警 → 写入 ClickHouse
- 离线分析: Airflow 调度 Spark (PySpark) ETL → 数仓 (Hive)
- 热点数据缓存: 设备最新状态存 Redis Hash, QPS > 10K
- 设备管理与 OTA 升级
 - 设备影子 (Device Shadow): JSON Patch 同步期望/报告状态 (基于 PostgreSQL JSONB)
 - 升级包分发: CDN + 断点续传 (Range Request), 进度追踪 (WebSocket 推送)

✅ 成果:

- 系统稳定运行 18 个月, 0 重大故障 (P0 级)
- 数据延迟从分钟级降至秒级 (95% < 1.5s)
- 主导编写《IoT 数据接入规范 V2.1》, 被 3 个新项目复用

🛒 电商平台订单履约系统重构 (技术 Owner) | 2020.06 – 2021.07

◆ 背景: 原系统耦合严重, 大促期间频繁超时 (2020 双 11 故障 3 次)

🔧 重构策略与实施:

- 领域驱动设计 (DDD) 落地
 - 划分限界上下文: 订单创建 → 库存锁定 → 物流调度 → 支付回调
 - 核心聚合根: Order (含状态机: created → paid → allocated → shipped → completed)
 - 事件驱动: 订单状态变更 → 发布 OrderPaidEvent → 触发库存/物流服务
- 关键技术升级:
 - 数据库: MySQL 5.7 → PostgreSQL 12 (利用 JSONB 存储扩展属性、窗口函数优化报表)

- 分库分表: ShardingSphere-JDBC (按 order_id 哈希), 支持平滑扩容
- 事务一致性: Saga 模式 (补偿事务) + 本地消息表 (最终一致性)
- 性能与稳定性:
 - 缓存策略: 多级缓存 (本地 Caffeine + Redis), 缓存击穿防护 (互斥锁 + 逻辑过期)
 - 压测优化: JMeter 全链路压测 → 定位慢 SQL (pg_stat_statements) → 索引优化 + 查询重写
 - 大促保障: 全链路压测 + 熔断降级 (Hystrix-like) + 资源隔离 (K8s QoS)

✅ 结果:

- 双 11 (2021) 零故障, 峰值 TPS 4,200 (提升 3.5 倍)
 - 订单创建平均耗时从 850ms → 120ms
 - 方案入选集团《高可用系统建设指南》案例
-

🎓 教育背景

浙江大学 | 软件工程 | 硕士

2017.09 – 2020.06

- 研究方向: 分布式系统一致性 (毕业论文:《基于 Raft 的日志复制优化》)
- GPA: 3.78/4.0

华中科技大学 | 计算机科学与技术 | 本科

2013.09 – 2017.06

📜 专业认证与影响力

- AWS Certified Developer – Associate (2024)
- 阿里云 ACA 云计算工程师 (2023)
- 开源贡献:
 - django-rest-framework-camel-case: PR #142 (支持嵌套序列化器配

置), 被合并

- celery-redbeat: 提交 Redis Cluster 兼容性修复
 - 技术分享:
 - QCon Shanghai 2024: 《Python 微服务在 SaaS 场景的落地挑战》
 - 公司内部: 主导“Python 性能调优”系列培训 (6 期, 覆盖 50+工程师)
-

✦ 自我评价

- **工程严谨:** 坚持“测试先行”, 核心模块单元测试覆盖率 $\geq 85\%$, CI 强制卡点
- **技术前瞻性:** 持续跟踪 async/await 生态 (探索 FastAPI + SQLAlchemy 2.0 async)
- **业务敏感:** 能从业务指标 (如 GMV、留存率) 反推技术优化方向
- **协作高效:** 熟练使用 Confluence/Jira/飞书, 技术方案文档平均阅读完成率 92%